ADA 19. oktober 2026

Sortering

Introducerende antagelse

Sortering i det virkelig liv finder sted på alle mulige forskellige datatyper.

For at undgå begrebsforvirring arbejder vi udelukkende med sortering af heltal (integers), under den antagelse, at alle de beskrevne principper kan anvendes for alle andre datatyper.

Inversion

Lad os betragte denne liste af heltal:

34, 8, 64, 51, 32, 21

Tallene er helt klar usorterede, så lad os prøve at se, om vi kan finde et mål for, hvor usorterede de er.

Til at hjælpe os definerer vi et begreb kaldet *inversion*. En inversion er et ordnet talpar (i,j), hvorom det gælder, at i < j, men a[i] > a[j].

I vores eksempel forekommer der følgende inversions:

(34,8) (34,32) (34,21) (64,51) (64,32) (64,21) (51,32) (51,21) (32,21)

Sortering

- Jo flere inversions listen indeholder, desto mere usorteret er den.
- Sorteringsdisciplinen går ud på at reducere antallet af inversions i en liste til nul.

Mere om inversions

- Hvad er antallet af inversions i denne liste: 78,63,61,45,25,11?
- Det gennemsnitlige antal inversions i en liste bestående af N forskellige heltal er N(N-1)/4.
- Hvis én operation kan eliminere én inversion, så må den slags sortering have kvadratisk tidskompleksitet $O(N^2)$.
- Hvis mere end én inversion kan elimineres af én operation, kan vi opnå bedre resultater end kvadratisk tidskompleksitet.

$O(N^2)$

Insertionsort

Selectionsort

Bubblesort

Bubblesort

```
1   public void bubbleSort(int array[]) // Java
2   {
3           for (int i = 0; i < array.length - 1; i++)
4           {
5                   boolean swapped = false;
6                   for (int j = 0; j < array.length – i - 1; j++)
7                   {
8                           if (array[j] > array[j+1])
9                           {
10                                  swap(array,j,j+1);
11                                  swapped = true;
12                          }
13                  }
14                  if (!swapped)
15                          return;
16          }
17  }
```

O(N logN)

Heapsort

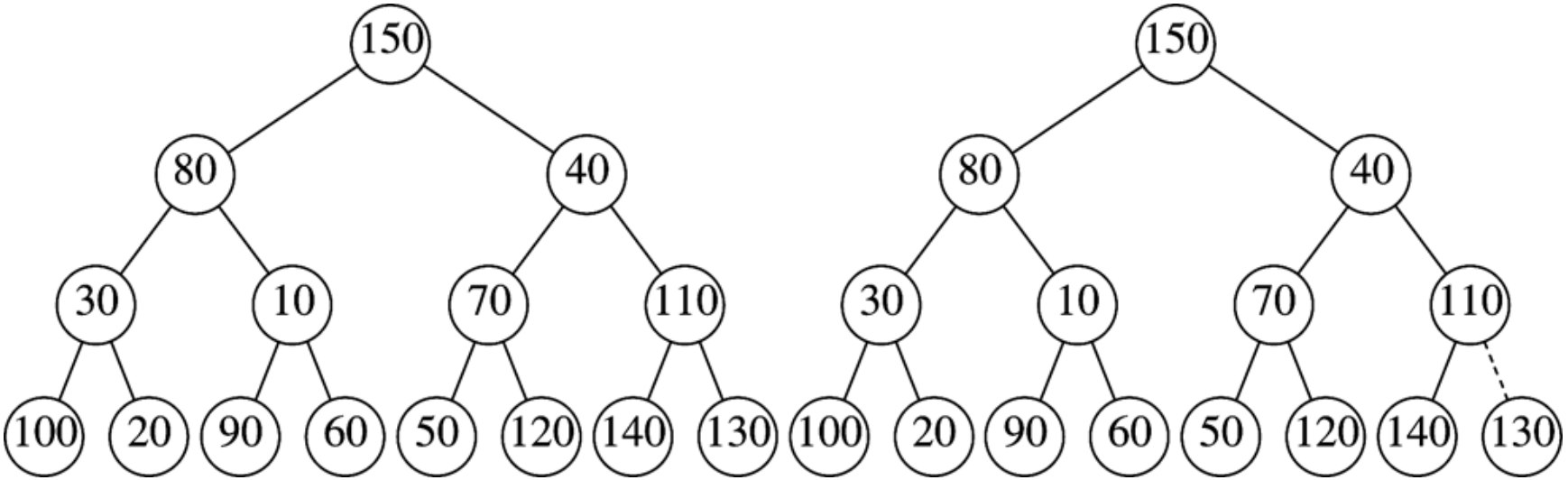
Mergesort

Quicksort

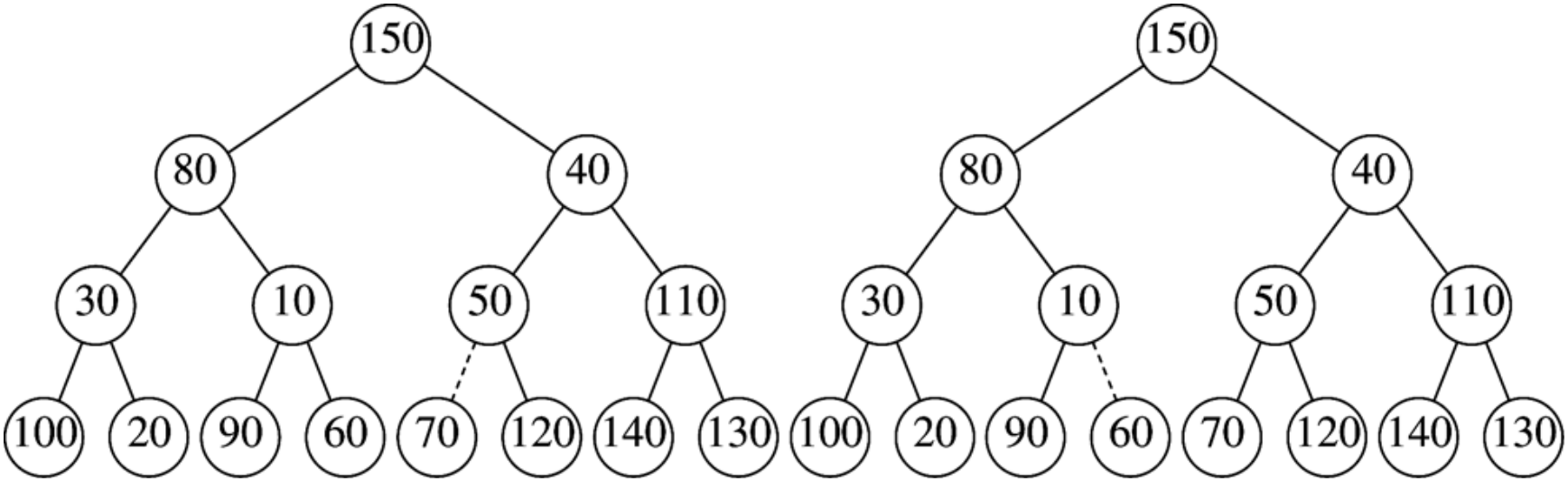
Heapsort og `buildHeap()`

- Idé: hvis vi har en tabel af usorterede tal, som vi kan lave om til en prioritetskø i lineær tid, kan vi sortere tabellen i O(N log N) tid.
- Antal operationer: N + N*log N -> O(N log).
- Lave et binært træ (array minus indeks 0) af samtlige indgående elementer.
- Kald `percolateDown()` for all noder under roden. Begynd med *rightmost deepest* som ikke er et blad.

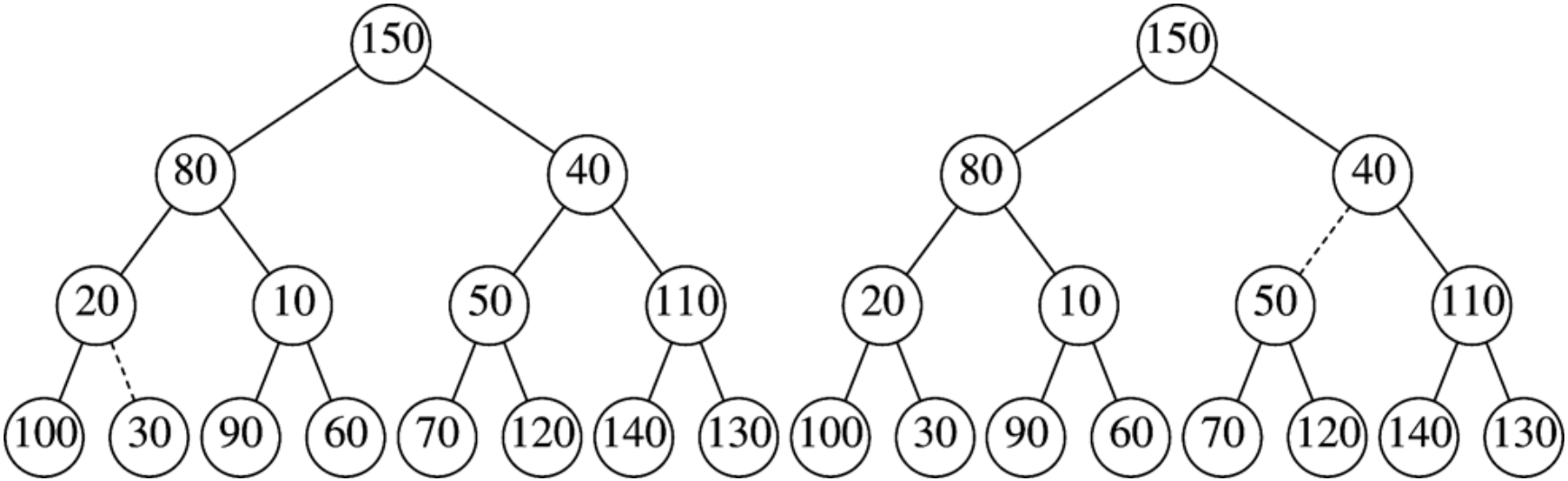
percolateDown(7)



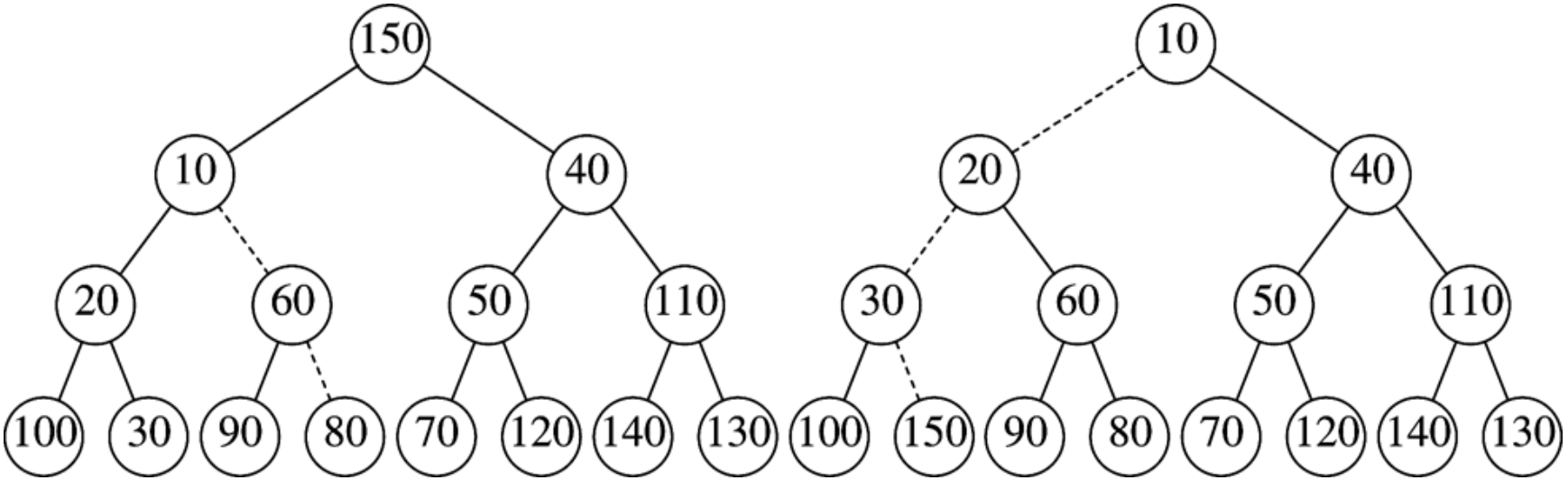
percolateDown(6) & (5)



percolateDown(4) & (3)



percolateDown(2) & (1)



`buildHeap()`

- Påstand: `buildHeap()` er O(N).
- Dette kan bevises ved at finde den øvre grænse for stiplede linjer i de foregående figurer.
- Som må være lig med summen af højderne af samtlige træets noder.
- Et perfekt binært træ med højden *h* har $2^{h+1} - 1$ noder, hvilket medfører at N = $2^{h+1} - 1$
- Bevis at summen af nodernes højder er $2^{h+1} - 1 - (h+1)$
- Derfor er summen af højderne og antallet af noder lineært forbundne.
- Summen af højderne er mindre end N.

Bevis

$$S = \sum_{i=0}^{h} 2^i \, (h - i)$$

= h+2(h-1)+4(h-2)+8(h-3)….+ $2^{h-1}(1)$

2S= 2h+4(h-1)+8(h-2)+16(h-3)…+ $2^h(1)$

S = -h+2+4+8+…+ $2^{h-1} + 2^h$

S = $2^{h+1} - 1 - (h+1)$

Hvorfor er *buildHeap*() vigtig?

- Sortering er en meget vigtig øvelse.
- Trivielle sorteringsalgoritmer har kvadratisk tidskompleksitet.
- Med buildHeap() kan man opnå O(N log N).
- Først bygger man en heap (prioritetskø) af sine usorterede tal. Det kræver max. N operationer.
- Så kaldes deleteMin() på alle elementer. Det tager N log N operations. Og tallene er sorterede.
- N + N log(N) -> O(N log N).

Mergesort

```
private static <AnyType extends Comparable<? super AnyType>>
void mergeSort( AnyType [ ] a, AnyType [ ] tmpArray, int left, int right )
{
    if( left < right )
    {
        int center = ( left + right ) / 2;
        mergeSort( a, tmpArray, left, center );
        mergeSort( a, tmpArray, center + 1, right );
        merge( a, tmpArray, left, center + 1, right );
    }
}

/**
 * Mergesort algorithm.
 * @param a an array of Comparable items.
 */
public static <AnyType extends Comparable<? super AnyType>>
void mergeSort( AnyType [ ] a )
{
    AnyType [ ] tmpArray = (AnyType[]) new Comparable[ a.length ];

    mergeSort( a, tmpArray, 0, a.length - 1 );
}
```

Merge

```
// Precondition: no duplicates
public static int[] merge (int[] a, int[] b)  //Java
{
    int [] m = new int[a.length+b.length];
    int pointerA = 0; int pointerB = 0; int pointerM = 0;

    while (pointerA < a. length && pointerB < b.length)
        if (a[pointerA] < b[pointerB])
            m[pointerM++] = a[pointerA++];
        else
            m[pointerM++] = b[pointerB++];

    while (pointerA < a.length)
        m[pointerM++] = a[pointerA++];

    while (pointerB < b.length)
        m[pointerM++] = b[pointerB++];

    return m;
}
```

Heapsort

Pladsproblemet kan løses ved at genbruge tabellen fra enden af.

Analysen er besværlig; men performance er relativ konstant.

Det gennemsnitlige antal sammenligninger er (muligvis ☺):

$$2N \log N - O(N \log \log N)$$

Mergesort

Er den sortering med de færreste sammenligninger og anvendes af Javas standardbibliotek.

C++ bruger QuickSort.

Analyse: best case, average case, worst case.

Shellsort

Shellsort er en 'multi-pass' algoritme. Hvert pass er en insertion sort af sekvenser bestående af hvert *h.* element for et for et fast (fixed) mellemrum *h* (også kaldt inkrementet). Dette kaldes *h*-sortering.

Analysen er besværlig. Simuleringer antyder best-case på $O(N^{5/4})$; men det kan ikke bevises.

Worst case er $O(N^2)$.

Shellsort - eksempel

Orig.	**81**	**94**	**11**	**96**	**12**	**35**	**17**	**95**	**28**	**58**	**41**	**75**	**15**
5	35	17	11	28	12	41	75	15	96	58	81	94	95
3	28	12	11	35	15	41	58	17	94	75	81	96	95
1	11	12	15	17	28	35	41	58	75	81	94	95	96

	0	1	2	3	4	5	6	7	8	9	10	11	12
0, 5, 10		1, 6, 11			2, 7, 12				3, 8			4, 9	
81,35,41		94,17,75			11,95,15				96,28			12,58	

Shellsort - eksperimenter

N = 1024

Shellsort	58.893	$N^{1,58}$
Bubblesort	261.563	$N^2/4$
1024^2	1.048.56	

Shellsort

- Som eksemplet illustrerer, er de subarrays, som Shellsort opererer på små til at starte med; senere bliver de større, men næsten sorterede. I begge til fælde er insertion sort effektiv.
- Shellsort is ustabil: det kan forekomme, at den ændrer rækkefølgen af elementer med samme værdi.
- Den har en 'naturlig' adfærd (lige som bubblesort), eftersom den afvikles hurtigere, hvis tabellen er relativt sorteret (få inversions).

bucketSort()

- Under særlige omstændigheder kan vi gøre det bedre end O(N log N).
- Hvis vi skal sortere positive heltal med en maksimumsværdi M, kan vi løse problemet ved først at lave et array med plads til M elementer.
- Tallene læses fra ende til anden, og når værdien X læses, så inkrementeres M[X] med en.
- Når alle tal er læst, kan det sorterede array etableres i overensstemmelse med antallet af forekomster i indekserne.
- Tidskomplekstet O(M+N) -> O(N).

Sortering

- Den nedre grænse for sortering, hvis der kun anvendes sammenligninger, er: $\Omega(N \log N)$.
- Det kan vises ved at tilføje alle sammenligninger til et binært træ (sektion 7.8).
- Der er forskel på *lower bound* and *best case*.

Konklusion

- Inversions
- $O(N^2)$ – bubble, insertion, selection
- O(N log N) – heap, quick, merge
- O(N) - bucket
- Vælg den mest hensigtsmæssige på baggrund af det talmateriale, der skal sorteres.
- Problemet er næppe vigtigt, hvis der er færre end 100.000 ikke alt for store objekter, og sorteringen derfor kan udføres i memory.

Animated Hungarian QuickSort

https://www.youtube.com/watch?v=ywWBy6J5gz8

Exercises

1. En tabel indeholder N sorterede heltal. Opgaven går ud på at afgøre om tabellen indeholder to tal, hvis sum er lig med parameteren X. Metoden skal skrives i to udgaver: én med kvadratisk tidskompleksitet og en med lineær.
2. Ordene *stale og least* er anagrammer, dvs. de indeholder nøjagtig de samme karakterer. Skriv en metode, som kan afgøre, om to ord (strings) er anagrammer.

Øvelser fra lærebogen med stigende sværhedsgrad.

7.44

7.38

7.43*

7.40*

Mergesort

```
private static <AnyType extends Comparable<? super AnyType>>
void mergeSort( AnyType [ ] a, AnyType [ ] tmpArray, int left, int right )
{
    if( left < right )
    {
        int center = ( left + right ) / 2;
        mergeSort( a, tmpArray, left, center );
        mergeSort( a, tmpArray, center + 1, right );
        merge( a, tmpArray, left, center + 1, right );
    }
}

/**
 * Mergesort algorithm.
 * @param a an array of Comparable items.
 */
public static <AnyType extends Comparable<? super AnyType>>
void mergeSort( AnyType [ ] a )
{
    AnyType [ ] tmpArray = (AnyType[]) new Comparable[ a.length ];

    mergeSort( a, tmpArray, 0, a.length - 1 );
}
```

Hvad er rekurrens?

- En rekurrens (recurrence) er en ligning, der beskriver en funktion ud fra dens værdi på mindre input.
- Eksempel: $T(N) = 2T(N/2)+N$
- Bruges især til at analysere rekursive algoritmer.
- Viser, hvordan problemet nedbrydes i delproblemer.

Eksempel på rekurrens

- Eksempel fra en algoritme, der opdeler problemet i to halvdele og samler resultaterne på lineær tid:
- $T(N) = 2T(N/2) + N$
- To rekursive kald på størrelse N/2
- Plus lineært arbejde for at kombinere resultaterne.

Hvorfor er rekurrenser vigtige?

- Bruges til at forudsige køretiden for rekursive algoritmer.
- Hjælper med at forstå væksten i beregningstid, når input vokser.
- Typiske anvendelser:
- Merge Sort: $T(N) = 2T(N/2) + O(N)$
- Binær søgning: $T(N) = T(N/2) + O(1)$
- Tower of Hanoi: $T(N) = 2T(N-1) + O(1)$

Metoder til at løse rekurrenser

- 1. Substitutionsmetoden – gæt løsningen og bevis den med induktion.
- 2. *Rekursionstræ-metoden – visualisér arbejdet på hvert niveau og summer det.
- 3. Master-teoremet – direkte formel til mange divide and conquer-algoritmer.

Intuitionen bag rekurrens

- En rekurrens beskriver selv-lighed i beregningen.
- Hvert kald deler problemet i mindre versioner af sig selv.
- Når basis-tilfældet nås, stopper rekursionen.
- Summen af arbejdet i alle niveauer giver den samlede kompleksitet.

Teleskop-kikkert



Vi kan trække den ud og vi kan folde den sammen.

Telescoping

- På dansk *teleskopering* 😉
- “En teleskoperende sum er en sum, hvor de fleste led går ud med hinanden.”
- Eller: “at forkorte en sum ved, at led går ud med hinanden”

Telescoping som bevismetode

- Først opstiller vi rekurrens-ligningen.
- Fx: $T(N) = 2T(N/2) + N$ (merge-sort)
- Vi stater med at dividere med N på begge sider.
- Herefter opstiller vi en ny ligning, hvor N erstattes med N/2.
- Hvis vi fortsætter med at lave nye ligninger, hvor N erstattes med N/2, får vi ligninger, hvor første udtryk på højresiden er lig venstresiden i næste ligning.
- Når vi kommer til bunds, dvs. til base-case, lægger vi alle venstresider og alle højresider sammen.
- Herefter går ovennævnte udtryk ud med hinanden.
- Til sidst har vi derfor kun venstresiden af første ligning tilbage, og den er lig med højresiden af sidste lignings første udtryk plus summen af højresidernes sidste udtryk.
- Tidskompleksiteten kan nu beregnes ved at gange igennem med N.

Analyse af MergeSort

$T(N) = 2T(N/2) + N$

$T(N)/N = T(N/2)/(N/2) + 1$

$T(N/2)/(N/2) = T(N/4)/(N/4) + 1$

$T(N/4)/(N/4) = T(N/8)/(N/8) + 1$

….. fortsæt til base-case.

$T(2)/2 = T(1)/1 + 1$

..... læg højre- og venstresider sammen og reducér.

$T(N)/N = T(1)/1 + \log N$

$T(N) = N + N \log N \rightarrow O(N \log N)$

Teleskop-kikkert



Vi kan trække den ud og vi kan folde den sammen.

Quicksort

```
1   public void quickSort(int array[], int left, int right)
2   {
3           if (left + CUTOFF < right)
4           {
5                   int pivot = median3(array, left, right);
6                   int i = left, j = right -1;
7                   for ( ; ; )
8                   {
9                           while( array[i] < pivot) i++;
10                          while( array[j] > pivot) j--;
11                          If (i < j)
12                                  swap(array, i, j);
13                          else
14                                  break;
15                  }
16                  swap(array, i, right - 1);
17
18                  quickSort(array, left, i - 1);
19                  quickSort(array, i + 1, right);
20          }
21          else
22                  insertionSort(array, left, right);
23  }
```

Analyse af worst case QuickSort

T(N) = T(N-1) + cN

T(N-1) = T(N-2) + c(N-1)

T(N-2) = T(N-3) + c(N-2)

T(N-3) = T(N-4) + c(N-3)

…. fortsæt til base case

T(2) = T(1) + c(2)

…… læg højre- og venstresider sammen og reducér.

T(N) = T(1) + c$\sum_{i=2}^{N} i$ -> $\theta(N^2)$